

Strings

Aho

```
// N * sqrt(sum of lens) ---- vaild to 5e5 it will be ~ 3e8
// if it all uniueq it will be ( n ^ 2)
struct Aho {
    int n;
    const int Alpha = 26;
    vector<vector<int> > nxt, out;
    vector<int> link, outlink;
    // nxt[][] at each node it tells u where to fail if i add that char
    // link[] at each node the lps
    // outlink[] at each node the lps i can match and has a pattern
    // out [][] at each node the indcies for the patterns i match it
    int node() {
        link.emplace_back(0);
        outlink.emplace_back(0);
        nxt.emplace_back(Alpha, 0);
        out.emplace_back(0);
        return n++;
    }

    Aho() : n(0) {
        node();
    };

    int idx(char c) {
        return c - 'a';
    }

    void add_pat(string &pat, int i) {
        int u = 0;
        for (auto c: pat) {
            if (nxt[u][idx(c)] == 0) nxt[u][idx(c)] = node();
            u = nxt[u][idx(c)];
        }
        out[u].emplace_back(i);
    }

    void build() {
        queue<int> q;
        for (q.push(0); !q.empty();) {
```

```

        int u = q.front();
        q.pop();
        for (int c = 0; c < Alpha; ++c) {
            int v = nxt[u][c];
            if (!v) nxt[u][c] = nxt[link[u]][c];
            else {
                link[v] = u ? nxt[link[u]][c] : 0;
                outlink[v] = out[link[v]].empty() ? outlink[link[v]] :
link[v];
                q.push(v);
            }
        }
    }
}

int go(int u, char c) {
    while (u and !nxt[u][idx(c)]) u = link[u];
    u = nxt[u][idx(c)];
    return u;
}
};

```

other

```

struct Aho {
    struct Node {
        array<int, 26> nxt;
        int link;
        vector<int> out;

        Node() {
            nxt.fill(-1);
            link = -1;
        }
    };

    vector<Node> t;
    Aho() { t.emplace_back(); }
    int add_string(const string &s, int id) {
        int v = 0;
        for (char c: s) {
            int x = c - 'a';
            if (t[v].nxt[x] == -1) {
                t[v].nxt[x] = t.size();
                t.emplace_back();
            }

```

```

        v = t[v].nxt[x];
    }
    t[v].out.push_back(id);
    return v;
}

void build() {
    queue<int> q;
    for (int c = 0; c < 26; c++) {
        int u = t[0].nxt[c];
        if (u != -1) {
            t[u].link = 0;
            q.push(u);
        } else t[0].nxt[c] = 0;
    }
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int c = 0; c < 26; c++) {
            int u = t[v].nxt[c];
            if (u != -1) {
                t[u].link = t[t[v].link].nxt[c];
                for (int id: t[t[u].link].out) t[u].out.push_back(id);
                q.push(u);
            } else {
                t[v].nxt[c] = t[t[v].link].nxt[c];
            }
        }
    }
}
};

```

Binary Trie

```

struct BinaryTrie {
    struct Node {
        int child[ALPHA] = {};
        int f = 0;
    };

    vector<Node> trie;

    BinaryTrie() {
        trie.emplace_back();
    }
}

```

```

void insert(int x) {
    int node = 0;
    for(int bit = LG; bit >= 0; bit--) {
        int val = (((1<<bit)&x) != 0);
        if(!trie[node].child[val]) {
            trie[node].child[val] = trie.size();
            trie.emplace_back();
        }
        node = trie[node].child[val];
        trie[node].f++;
    }
}

void erase(int x) {
    int node = 0;
    for(int bit = LG; bit >= 0; bit--) {
        int val = (((1<<bit)&x) != 0);

        node = trie[node].child[val];
        trie[node].f--;
    }
}

int query(int x) {
    int node = 0;
    int ret = 0;
    for(int bit = LG; bit >= 0; bit--) {
        int val = (((1<<bit)&x) == 0);
        if(trie[trie[node].child[val]].f)
            ret |= (1<<bit);
        else
            val ^= 1;

        node = trie[node].child[val];
        trie[node].f++;
    }
}
};

```

Hash

```

#include <bits/stdc++.h>
#define el '\n'
#define ll long long

```

```

#define ld long double
#define ToshToshToshToshTosh ios_base::sync_with_stdio(0); cin.tie(0);
cout.tie(0);
const int mod= 1e9 +7, N = 2e3+4;
using namespace std;
const int B = 2;
const int b[] = {200003, 200009};
int pw[B][N], inv[B][N], invB[B];
int r[B];
#define multihash array<int, B>
int add(int a, int b) {
    return ((a+=b) < mod? a : a-mod);
}
int sub(int a, int b) {
    return ((a-=b) < 0? a + mod : a);
}
int mul(int a, int b) {
    return (1ll * a * b)%mod;
}
int fp(int b, int e) {
    if (!e)
        return 1;
    int ret = fp(b, e >> 1);
    ret = mul(ret, ret);
    return (e&1?mul(ret, b) : ret);
}

void pre() {
    auto now = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::microseconds>
(now.time_since_epoch());
    srand(duration.count());
    for (int i = 0; i < B; ++i) {
        pw[i][0] = inv[i][0] = 1;
        invB[i] = fp(b[i], mod-2);
        r[i] = rand() + 1;
    }
    for (int i = 1; i < N; ++i)
        for (int base = 0; base < B; ++base)
            pw[base][i] = mul(pw[base][i-1], b[base]), inv[base][i] =
mul(inv[base][i-1], invB[base]);
}

struct multisethash {
    vector<multihash> h;
    vector<multihash> hinv;
}

```

```

vector<int> v;
int n;
multisetHash(vector<int> &v) {
    this->v = v;
    n = v.size();
    h.resize(n + 1);
    hinv.resize(n + 1);
    for (int base = 0; base < B; ++base)
        h[0][base] = 1, hinv[0][base] = 1;
    for (int i = 1; i <= n; ++i) {
        int cur = v[i - 1];
        for (int j = 0; j < B; ++j) {
            h[i][j] = mul(h[i - 1][j], add(cur, r[j]));
            hinv[i][j] = fp(h[i][j], mod-2);
        }
    }
}

multisetHash get_hash(int l, int r) {
    multisetHash ret;
    for (int i = 0; i < B; ++i) {
        ret[i] = mul(h[r][i], hinv[l-1][i]);
    }
    return ret;
}

};

struct Hash {
    vector<multisetHash> h, rev;
    vector<int> s;
    int n;
    int hash_num(int x, int i, int base) {
        return mul(x, pw[base][i]);
    }
    Hash(vector<int> &s) {
        this->s = s;
        n = s.size();
        h.resize(n+1);
        rev.resize(n+2);
        for (int i = 1; i <= n; i++)
            for (int base = 0; base < B; ++base)
                h[i][base] = add(h[i-1][base], hash_num(s[i-1], i, base));

        for (int i = n; i >= 1; i--) {
            for (int base = 0; base < B; ++base)
                rev[i][base] = add(rev[i+1][base], hash_num(s[i-1], n - i + 1,
base));

```

```

    }
}

multihash get_hash(int l, int r) {
    multihash ret;
    for (int base = 0; base < B; ++base) {
        ret[base] = mul(subt(h[r][base], h[l-1][base]), inv[base][l-1]);
    }
    return ret;
}

multihash get_rev_hash(int l, int r) {
    multihash ret;
    for (int base = 0; base < B; ++base)
        ret[base] = mul(subt(rev[l][base], rev[r+1][base]), inv[base][n -
r]);
    return ret;
}

int getpos(Hash &s, vector<int> &pos) {
    int ret = pos.front();
    int n = pos.size();
    for (int i = 1; i < n; i++) {
        int l = 0, r = (int) s.s.size() - pos[i] - 1, mx = r, md{};
        while (l <= r) {
            md = l + r >> 1;
            if (s.get_hash(ret + 1, ret + md + 1) == s.get_hash(pos[i] + 1,
pos[i] + md + 1))
                l = md + 1;
            else r = md - 1;
        }
        int dis = l - 1;
        if (dis == (mx)) continue;
        if (s.s[ret + dis + 1] < s.s[pos[i] + dis + 1]) ret = pos[i];
    }
    return ret;
}

};

void solve() {
    int n;
    cin >> n;
    vector<pair<int, int>> a(n);
    vector<int> b;
    for (auto &[f, s] : a) {
        cin >> f >> s;
        if (f > s)
            swap(f, s);
    }
}

```

```

        b.push_back((f << 13) | s);
    }
    multiset hash hshb(b);
    map<multiset, int> s;
    int cnt{};

    for (int i = 1; i <= n; i++) {
        for (int j = i; j <= n; j++) {
            auto hsh = hshb.get_hash(i, j);
            s[hsh]++;
        }
    }
    for (auto &[hsh, nm]:s)
        cnt += (nm * nm - nm >> 1);
    cout << cnt << endl;
}

signed main() {
    pre();
    ToshToshToshToshTosh
    int t = 1;
    cin >> t;
    int C = 1;
    while (t--)
        solve();
    return 0;
}

```

Mancher

```

void man(string &s) // distinct number of oalis at most n
{
    int n = (int)s.size();
    vector<int> d1(n);

    for(int i=0 , l=0 , r=-1; i<n; i++)
    {
        int k = (i>r) ? 1 : min(d1[r-i+l] , r-i+1);

        while(0 <= i-k && i+k < n && s[i-k]==s[i+k])
            k++;

        d1[i] = k--;
        if (i+k > r)
        {

```

```

        l = i-k;
        r = i+k;
    }
}

vector<int> d2(n);

for(int i=0 , l=0 , r=-1; i<n; i++)
{
    int k = (i>r) ? 0 : min(d2[r-i+l+1] , r-i+1);

    while(0 <= i-k-1 && i+k < n && s[i-k-1]==s[i+k])
        k++;

    d2[i] = k--;
    if (i+k > r)
    {
        l = i-k-1;
        r = i+k;
    }
}

}

auto ispali = [&](int l,int r)-> bool {
    int len = r - l + 1;
    int mid = (l + r) / 2;
    if (len & 1) return p1[mid] >= (len + 1) / 2;
    return p2[mid + 1] >= len / 2;
};

```

String matching using bitset

```

#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 9;
vector<int> v;
bitset<N>bs[26], oc;
int main() {
    int i, j, k, n, q, l, r;
    string s, p;
    cin >> s;
    for(i = 0; s[i]; i++) bs[s[i] - 'a'][i] = 1;
    cin >> q;
    while(q--) {

```

```

cin >> p;
oc.set();
for(i = 0; p[i]; i++) oc &= (bs[p[i] - 'a'] >> i);
cout << oc.count() << endl; // number of occurrences
int ans = N, sz = p.size();
int pos = oc._Find_first();
v.push_back(pos);
pos = oc._Find_next(pos);
while(pos < N) {
    v.push_back(pos);
    pos = oc._Find_next(pos);
}
for(auto x : v) cout << x << ' '; // position of occurrences
cout << endl;
v.clear();
cin >> l >> r; // number of occurrences from l to r, where l and r is 1-
indexed
if(sz > r - l + 1) cout << 0 << endl;
else cout << (oc >> (l - 1)).count() - (oc >> (r - sz + 1)).count() <<
endl;
}
return 0;
}

```

Suffix array

```

// NLog(N) , 1->Based
struct SuffixArray {
    int n;
    // p -> Index Of Sorted Suffixes (0->Based)
    // p[0] = index of the smallest suffix -> $
    // p[1] = index of the second smallest suffix
    // c -> Class Of Each Suffix (For Comparing) : Idx Of Suffix In P
    // lcp -> Size of max prefix match with suffix (i - 1)
    vector<int> p, c, lcp;
    string s;

    SuffixArray(string s) {
        s += '\0', this->s = s, n = s.size();
        p = c = lcp = vector<int>(n);
        for (int i = 0; i < n; i++) p[i] = i;
        sort(p.begin(), p.end(), [&](int a, int b) { return s[a] < s[b]; });
    }
    // if descending >
    for (int i = 1; i < n; i++)
        c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
}

```

```

    int k = 0;
    while ((1 << k) < n) {
        for (int i = 0; i < n; i++)
            p[i] = (p[i] - (1 << k) + n) % n;
        vector<int> idx(n), nc(n), temp = p;
        for (auto x: c) idx[x]++;
        for (int i = 1; i < n; i++) idx[i] += idx[i - 1];
        for (int i = n - 1; i >= 0; i--)
            p[--idx[c[temp[i]]]] = temp[i];
        for (int i = 1; i < n; i++)
            nc[p[i]] = nc[p[i - 1]] +
                (make_pair(c[p[i]], c[(p[i] + (1 << k)) % n]) !=
                 make_pair(c[p[i - 1]], c[(p[i - 1] + (1 << k)) %
n]]));

        c = nc;
        k++;
    }
    k = 0;
    for (int i = 0; i < n - 1; i++) {
        while (s[i + k] == s[p[c[i] - 1] + k]) k++;
        lcp[c[i]] = k, k = max(k - 1, 0);
    }
}

string getKthSuffix(int k) {
    return s.substr(p[k], s.size() - p[k] - 1) + '$';
}
};

```

other

```

void countsort(vector<int> &p, vector<int> &c) {
    int n = p.size();
    vector<int> cnt(n, 0);
    for (auto x: c) cnt[x]++;
    vector<int> p_new(n);
    vector<int> pos(n);
    pos[0] = 0;
    for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
    for (auto x: p) {
        int i = c[x];
        p_new[pos[i]] = x;
        pos[i]++;
    }
    p = p_new;
}

```

```

}

vector<int> suffixarray(vector<int> v) {
    vector<int> s = v;
    s.push_back(-2e9);
    int n = s.size();
    vector<int> p(n), c(n);
    {
        vector<pair<int, int> > a(n);
        for (int i = 0; i < n; i++) a[i] = {s[i], i};
        sort(a.begin(), a.end());
        for (int i = 0; i < n; i++) p[i] = a[i].second;
        c[p[0]] = 0;
        for (int i = 1; i < n; i++) {
            if (a[i].first == a[i - 1].first) c[p[i]] = c[p[i - 1]];
            else c[p[i]] = c[p[i - 1]] + 1;
        }
    }

    int k = 0;
    while ((1 << k) < n) {
        for (int i = 0; i < n; i++) p[i] = (p[i] - (1 << k) + n) % n;
        countsort(p, c);
        vector<int> c_new(n);
        c_new[p[0]] = 0;
        for (int i = 1; i < n; i++) {
            pair<int, int> prv = {c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]};
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << k)) % n]};
            if (prv == cur) c_new[p[i]] = c_new[p[i - 1]];
            else c_new[p[i]] = c_new[p[i - 1]] + 1;
        }
        c = c_new;
        k++;
    }
    return p;
}

vector<int> lcp(vector<int> s, vector<int> &p) {
    int n = p.size();
    vector<int> rank(n);
    for (int i = 0; i < n; i++) rank[p[i]] = i;
    int k = 0;
    vector<int> lcp(n - 1);
    for (int i = 0; i < n - 1; i++) {
        if (rank[i] == n - 1) {
            k = 0;

```

```

        continue;
    }
    int j = p[rank[i] + 1];
    while (i + k < n - 1 and j + k < n - 1 and s[i + k] == s[j + k])k++;
    lcp[rank[i]] = k;
    if (k)k--;
}
return lcp;
}

```

Suffix automaton

```

#include <bits/stdc++.h>
#define ll long long
#define ld long double
using namespace std;
const ll N = 1e5 + 10, LG = 20, mod = 1e9 + 7;
struct SuffixAutomaton
{
    int last = 0, cntState = 1;
    struct state
    {
        int len = 0, link = -1, cnt = 0, first_pos = 0;
        map<char, int> next;
    };
    state st[N * 2];
    ll dp[N * 2];
    void addChar(char c)
    {
        int p = last;
        int cur = cntState++;
        st[cur].len = st[p].len + 1;
        st[cur].cnt = 1;
        st[cur].first_pos = st[p].len - 1;
        while(p != -1 && st[p].next.count(c) == 0)
        {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if(p == -1)
        {
            st[cur].link = 0;
        }
        else
        {

```

```

        int q = st[p].next[c];
        if(st[q].len == st[p].len + 1)
        {
            st[cur].link = q;
        }
        else
        {
            int clone = cntState++;
            st[clone].link = st[q].link;
            st[clone].len = st[p].len + 1;
            st[q].link = st[cur].link = clone;
            st[clone].next = st[q].next;
            st[clone].cnt = 0;
            st[clone].first_pos = st[q].first_pos;
            while(p != -1 && st[p].next[c] == q)
            {
                st[p].next[c] = clone;
                p = st[p].link;
            }
        }
    }
    last = cur;
}

bool checkForOccurrence(string str)
{
    int cur = 0;
    for(int i = 0; i < str.size(); i++)
    {
        if(st[cur].next.count(str[i]))
        {
            cur = st[cur].next[str[i]];
        }
        else
            return false;
    }
    return true;
}

ll numberOfSubstrings()
{
    ll ans = 0;
    for(int i = 1; i < cntState; i++)
    {
        ans += st[i].len - st[st[i].link].len;
    }
    return ans;
}

```

```

ll lengthOfSubstrings()
{
    auto getSum = [&](ll x) -> ll {
        return x * (x + 1) / 2;
    };
    ll ans = 0;
    for(int i = 1; i < cntState; i++)
    {
        ans += getSum(st[i].len) - getSum(st[st[i].link].len);
    }
    return ans;
}

void numberOfOccPreProcess()
{
    static bool visisted = false;
    if(visisted)return;
    visisted = true;
    vector<pair<int, int>> v;
    for(int i = 0; i < cntState; i++)
        v.push_back(pair<int, int>(st[i].len, i));
    sort(v.rbegin(), v.rend());
    for(int i = 0; i < cntState; i++)
    {
        if(st[v[i].second].link > 0)
            st[st[v[i].second].link].cnt += st[v[i].second].cnt;
        dp[v[i].second] = st[v[i].second].cnt;
        for(auto x: st[v[i].second].next)
        {
            dp[v[i].second] += dp[x.second];
        }
    }
}

ll numberOfOcc(string s)
{
    numberOfOccPreProcess();
    int cur = 0;
    for(char c: s)
    {
        if(st[cur].next.count(c))
        {
            cur = st[cur].next[c];
        }
        else
            return 0;
    }
    return st[cur].cnt;
}

```

```

}
void dfs(int node, ll &k, string &ans)
{
    k -= st[node].cnt;
    if(k <= 0) return;
    for(auto x: st[node].next)
    {
        if(dp[x.second] < k)
        {
            k -= dp[x.second];
        }
        else
        {
            ans.push_back(x.first);
            dfs(x.second, k, ans);
            return;
        }
    }
}

string getKthLex(ll k)
{
    numberOfOccPreProcess();
    if(dp[0] < k)
        return "No such line.";
    string ans = "";
    dfs(0, k, ans);
    return ans;
}

};

string lcs (string S, string T) {
    SuffixAutomaton sf;
    for (int i = 0; i < S.size(); i++)
        sf.addChar(S[i]);

    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < T.size(); i++) {
        while (v && !sf.st[v].next.count(T[i])) {
            v = sf.st[v].link ;
            l = sf.st[v].len;
        }
        if (sf.st[v].next.count(T[i])) {
            v = sf.st[v].next[T[i]];
            l++;
        }
        if (l > best) {

```

```

        best = l;
        bestpos = i;
    }
}
return T.substr(bestpos - best + 1, best);
}
void burn()
{
    string s;
    cin >> s;
    string x;
    x.resize(s.size());
    for(int i = 0; i < s.size(); i++)
    {
        if(s[i] == 'A')
            x[i] = 'T';
        else if(s[i] == 'T')
            x[i] = 'A';
        else if(s[i] == 'C')
            x[i] = 'G';
        else
            x[i] = 'C';
    }
    reverse(x.begin(), x.end());
    auto res = lcs(s, x);
    cout << res.size() << '\n';
    cout << res << '\n';
}

signed main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        burn();
}

```

Trie

```

const int ALPHA = 26;

struct Trie {
    struct Node {
        int child[ALPHA] = {};
    };
};

```

```

        int f = 0;
    };

    vector<Node> trie;

    Trie() {
        trie.emplace_back();
    }

    void insert(string& s) {
        int node = 0;
        for(auto& i : s) {
            int chIdx = i-'a';
            if(!trie[node].child[chIdx]) {
                trie[node].child[chIdx] = trie.size();
                trie.emplace_back();
            }
            node = trie[node].child[chIdx];
            trie[node].f++;
        }
    }

    void erase(string& s) {
        int node = 0;
        for(auto& i : s) {
            int chIdx = i-'a';

            node = trie[node].child[chIdx];
            trie[node].f--;
        }
    }

    int query(string& s) {
        int node = 0;
        for(auto& i : s) {
            int chIdx = i-'a';
            if(!trie[trie[node].child[chIdx]].f)
                return 0;
            node = trie[node].child[chIdx];
            trie[node].f++;
        }
        return trie[node].f;
    }
};

```

Z and Kmp

```
// returns the longest proper prefix array of pattern p
// where lps[i]=longest proper prefix which is also suffix of p[0...i]
vector<int> build_lps(string p) {
    int sz = p.size();
    vector<int> lps;
    lps.assign(sz + 1, 0);
    int j = 0;
    lps[0] = 0;
    for(int i = 1; i < sz; i++) {
        while(j >= 0 && p[i] != p[j]) {
            if(j >= 1) j = lps[j - 1];
            else j = -1;
        }
        j++;
        lps[i] = j;
    }
    return lps;
}
```

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i-1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

```
vector<int>ans;
// returns matches in vector ans in 0-indexed
void kmp(vector<int> lps, string s, string p) {
    int psz = p.size(), sz = s.size();
    int j = 0;
    for(int i = 0; i < sz; i++) {
        while(j >= 0 && p[j] != s[i])
            if(j >= 1) j = lps[j - 1];
    }
```

```

        else j = -1;
    j++;
    if(j == psz) {
        j = lps[j - 1];
        // pattern found in string s at position i-psz+1
        ans.push_back(i - psz + 1);
    }
    // after each loop we have j=longest common suffix of s[0..i] which is
    // also prefix of p
}
}

// z(i) denotes the maximum length of a substring that begins at position i
// i and is a prefix of the string
vector<int> build_z(string s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n and s[z[i]] == s[z[i] + i]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}

```